

Semigroups & Aggregators

"How to Add Things"

Adam Laiacano
Spotify Data Guild L&L
April 12, 2017

1 / 22

Semigroups

Wikipedia:

Semigroup is an algebraic structure consisting of a *set* together with an *associative binary operation*.

- "set" = Scala datatype: Int, Double, List[T], etc.
- "associative binary operation" = Numeric addition, list concatenation

Two crucial properties:

$(1 + 2) + 3 = 1 + (2 + 3)$ associative! (required)

$1 + 2 = 2 + 1$ commutative! (good, not required)

```
// Adding two integers results in a new integer
def plus(a: Int, b: Int): Int = a + b

// Adding two lists is not commutative!
def plus[T](a: List[T], b: List[T]): List[T] = a ++ b
```

Making a Semigroup in Algebird

Definition:

```
trait Semigroup[T] {  
  /**  
   * Combines two `T` instances associatively.  
   * @return result of combining `l` and `r`  
   */  
  def plus(l: T, r: T): T  
}
```

Making your own Semigroup:

```
val maxValueSemigroup = Semigroup  
  .from { (l: Double, r: Double) => math.max(l, r) } // Sets the "plus" function  
  
maxValueSemigroup.plus(10.2, 10.3) // 10.3  
  
List(1.0, 2.0, 3.0)  
  .reduce(maxValueSemigroup.plus) // 3.0  
  
List(1.0, 2.0, 3.0)  
  .reverse  
  .reduce(maxValueSemigroup.plus) // Still 3.0 (commutative!)
```

3 / 22

Semigroup usage in scio

```
val input = List(  
  ("a", 1),  
  ("a", 2),  
  ("b", 1)  
)  
sc.parallelize(input)  
  .sumByKey  
  // List(("a", 3.0), ("b", 1.0))
```

```
def sumByKey(implicit sg: Semigroup[V]): SCollection[(K, V)] =  
  this.applyPerKey(Combine.perKey(Functions.reduceFn(sg)), kvToTuple[(K, V)])
```

Tell scio to use `maxValueSemigroup`

```
sc.parallelize(input)  
  .sumByKey(maxValueSemigroup)  
  // List(("a", 2.0), ("b", 1.0))
```

Use Moments for summary statistics!

Here's how two Moments are added:

```
import com.twitter.algebird.Moments
import com.twitter.algebird.Operators._

val combined = Moments(4) + Moments(10)
combined.count    // 2L
combined.mean     // 7.0
combined.variance // 9.0
// combined.stddev, combined.skewness, combined.kurtosis
```

5 / 22

Aggregators

"I'm amazed by how much they are able to do with that simple abstraction."

- Oscar Boykin, author of Algebird & Scalding

The fun stuff: Aggregators

Aggregating data is a way of reducing many datapoints into one number *per key* (eg streams per artist).

Familiar examples are: sum, mean, min, max.

At a higher level, we are performing three operations:

1. Transforming input data into a form that it can be easily *reduced* (eg `trackEntity.getPlayedMs`)
2. Combining the data in its reduceable form (using `Semigroup.plus`)
3. Transforming the data from its reduceable form to give a final, presentable result.

The fun stuff: Aggregators

Algebird aggregators have *three key datatypes*, and *two key functions*, and a Semigroup:

```
trait Aggregator[-A, B, +C]
```

Datatypes:

- **-A** - The input type (any subclass of A works == "contravariant")
- **B** - The internal state type.
- **+C** - The output presentation type (any superclass of C works == "contravariant")

Functions:

- **Aggregator.fromSemigroup** (or fromMonoid) - Sets the Semigroup[B]
- **Aggregator.composePrepare** - A function from A => B
- **Aggregator.andThenPresent** - A function from B => C (optional)

An example will help...

The fun stuff: Aggregators

Let's find the max value of a `List[Double]` using an `Aggregator`, and returns the result as an `Int`.

```
case class MaxValue(value: Double)

val maxValueSemi: Semigroup[MaxValue] = Semigroup.from {
  (l: MaxValue, r: MaxValue) => MaxValue(math.max(l.value, r.value))
}

val mvAggregator: Aggregator[Double, MaxValue, Int] = Aggregator
  // sets the value of 'semigroup'
  .fromSemigroup(maxValueSemi)

  // converts from input to internal state
  .composePrepare[Double](v => MaxValue(v))

  // converts from internal state to output, which we want to be an Int
  .andThenPresent(v => v.value.toInt)

mvAggregator(Seq(1,2,3,4, 10.3, 11.4, -3))
// 11
```

Aggregator usage in scio

```
val mvAggregator: Aggregator[Double, MaxValue, Int] = ...  
  
val input = List(  
  ("a", 1.2),  
  ("a", 2.4),  
  ("b", 1.0)  
)  
  
sc.parallelize(input)  
  .aggregateByKey(mvAggregator)  
  // List(("a", 2), ("b", 1))
```

10 / 22

Go time: aggregators in scio pipelines

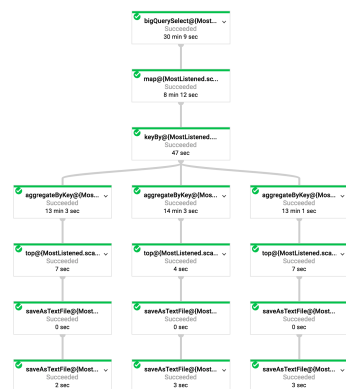
Goal: Extract three features from `TrackEntity`:

- Average track length per artist
- Number of tracks per artist
- Span of career for each artist (newest track - oldest track)

First up: [one aggregator per feature](#).

Go time: aggregators in scio pipelines

First up: one aggregator per feature.

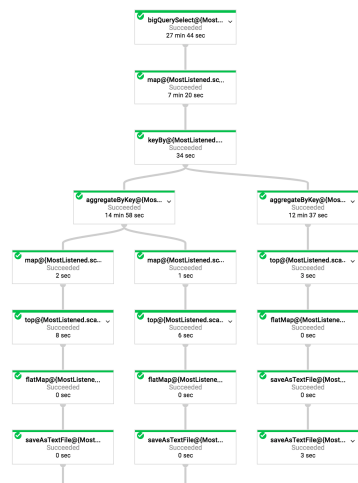


Go time: aggregators in scio pipelines

Take two: [combine Moments aggregators](#).

Go time: aggregators in scio pipelines

Take two: [combine Moments aggregators](#).



14 / 22

Go time: aggregators in scio pipelines

Winner: [MultiAggregator](#).

15 / 22

Go time: aggregators in scio pipelines

Winner: [MultiAggregator](#).



Thanks

laiacano@
@adamlaiacano

Some links:

- [Scio code from this presentation](#)
- [Algebird website & resources](#)

Some other links:

- [MAB implementation & Simulation](#)
- [Sifter](#) (small MAB library I wrote)

If there's time: implicits

```
// import com.twitter.algebird.Operators._  
val combined = Moments(4) + Moments(10)  
// Type mismatch, expected: String, actual: Moments
```

How does scala know how to add `Moments(4) + Moments(10)`?

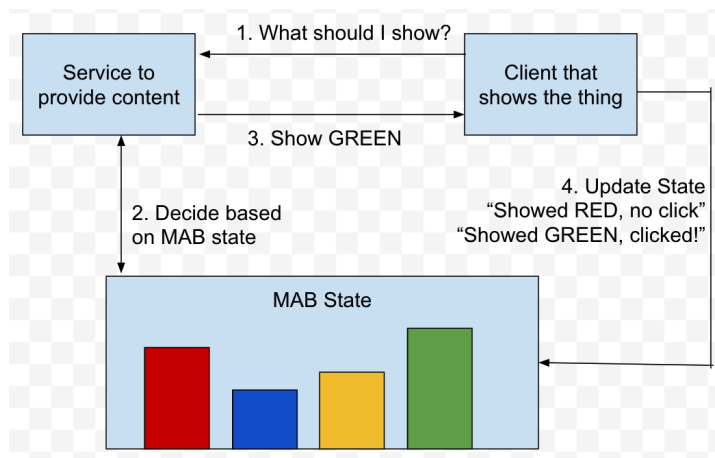
Operators contains conversions to implicit classes that override `+`, allowing us to use `Semigroup.plus` instead.

Here's the `+` that's being used to add Moments:

```
object Operators {  
  implicit def toPlus[T: Semigroup](t: T) = new PlusOp(t)  
  implicit def toMinus[T: Group](t: T) = new MinusOp(t)  
  implicit def toTimes[T: Ring](t: T) = new TimesOp(t)  
  implicit def toDiv[T: Field](t: T) = new DivOp(t)  
  implicit def toRichTraversableFromIterator[T](t: Iterator[T]): RichTraversable[T] =  
    new RichTraversable(t)  
  implicit def toRichTraversable[T](t: Traversable[T]): RichTraversable[T] =  
    new RichTraversable(t)  
}  
  
class PlusOp[T: Semigroup](t: T) {  
  def +(other: T) = implicitly[Semigroup[T]].plus(t, other)  
}
```

18 / 22

Oh, Multi-Armed Bandits



Multi-Armed Bandits 101: Decision

We'll use an **EpsilonGreedy** decision function. The logic for deciding which arm to show is:

- With probability ϵ , show the "arm" that performs the best.
- Otherwise, choose randomly between the available options.

Probability of an arm being shown is: $\epsilon * \mathbf{1}(\text{arm}) + (1 - \epsilon) / N$

Example:

With $\epsilon = 0.8 = 4/5$, and 3 arms:

- A(value=10)
- B(value=5)
- C(value=1)

what is the proportion that each is selected?

$$\begin{array}{lcl} \text{A: } 4/5 + (1 - 4/5) / 3 & = & 12/15 + 1/15 = 13/15 \\ \text{B: } 0 + (1 - 4/5) / 3 & & = 1/15 \\ \text{C: } 0 + (1 - 4/5) / 3 & & = 1/15 \end{array}$$

MAB Simulation

```
case class EpsilonGreedy(state: Seq[Arm], epsilon: Double = 0.9) { ... }
case class ExperimentResult(name: String, earnedValue: Double, nPulls: Int)
implicit val egSemi: Semigroup[EpsilonGreedy] = ...

val egAggregator: Aggregator[ExperimentResult, EpsilonGreedy, EpsilonGreedy] =
  Aggregator
    .fromSemigroup(egSemi)
    .composePrepare{(arm: ExperimentResult) => EpsilonGreedy(Seq(arm)) }

val allObservations: Map[String, Map[String, EpsilonGreedy]] = simulation
  .map { case (userId, country, genre, version, click) =>
    val result = ExperimentResult(version, click.toInt, 1)

    Map(
      "user" -> Map(userId -> egAggregator.prepare(result)),
      "genres" -> Map(genre -> egAggregator.prepare(result)),
      "country" -> Map(country -> egAggregator.prepare(result))
    )
  }
  .toList.reduce(_ + _) // "+" uses Operators.PlusOp to call egSemi.plus
```

Now we have the feedback for each user, as well as aggregated results for the country they live in and their favorite genre.

How to use it

View aggregated results for users who prefer pop, metal, punk or any combination.

```
val genreBandits = allObservations("genres")
val punkOrMetal = genreBandits("punk") + genreBandits("metal")

("punk",
 genreBandits("punk").totalPulls,
 genreBandits("punk").totalValue,
 genreBandits("punk").topArm)
("punk or metal",
 punkOrMetal.totalPulls,
 punkOrMetal.totalValue,
 punkOrMetal.topArm)
```

genre	pulls value		top version
pop	345	47.0	a
punk	326	180.0	b
metal	329	114.0	c
punk or metal	655	294.0	c